

EXERCISE SOLUTIONS



Module 4: Constraints, Database Design & Normalization

Module 4 - 8 Hours

Constraints, Database Design & Normalization

Notes for the trainer

Core and stretch. The questions marked **(Core)** are the required set that fits the session. Debrief those with the whole room. Cover the unmarked stretch questions only if the batch worked ahead, or leave them as take-home reading.

Release after the practice window. Marks total 100.

Section	Questions	Marks
Level 1 - Understanding	Q1 to Q10	30
Level 2 - Design and normalize	Q11 to Q21	45
Level 3 - Judgement	Q22 to Q27	25

Design questions have more than one defensible answer. Mark the reasoning, not the resemblance to this model answer.

Level 1 - Understanding

Q1 (Core)

Key type	Example on <code>loan</code>
Super key	<code>(loan_id, principal_amount)</code>
Candidate key	<code>loan_id</code>
Primary key	<code>loan_id</code>
Foreign key	<code>customer_id</code> , and separately <code>branch_id</code>

`(loan_id, principal_amount)` is a super key because it uniquely identifies a row, but it is not a candidate key because it is not **minimal**: remove `principal_amount` and it is still unique, so the extra column contributed nothing.

Q2 (Core)

Three reasons to refuse:

1. **Not stable.** People change email addresses. Every foreign key referencing the customer would have to change with it, across `account`, `loan`, `fixed_deposit` and `beneficiary`, inside one transaction.
2. **Nullable in reality.** Not every ICICI Bank customer has an email. A primary key cannot be NULL, so the bank could not onboard a customer without one.
3. **Wide and repeated.** An 80-character string copied into every child row costs space in the table, in every index on that column, and in every join comparison, against four bytes for an integer.

Instead: keep `customer_id` as a surrogate primary key and declare `UNIQUE (email)`. The business rule is still enforced by the engine, and identity stays stable.

Q3 (Core)

`RESTRICT` refuses the delete if children exist. `CASCADE` deletes the children too.

Relationship	Rule	Reason
<code>account_txn</code> on <code>account</code>	<code>RESTRICT</code>	Transaction history is regulated evidence, retained for years
<code>card_reward_enrolment</code> on <code>credit_card</code>	<code>CASCADE</code>	An enrolment has no meaning without the card it belongs to
<code>loan</code> on <code>customer</code>	<code>RESTRICT</code>	A customer with an outstanding loan cannot simply be removed
<code>employee.manager_id</code>	<code>SET NULL</code>	When a manager leaves, the team must survive, unassigned
<code>loan_payment</code> on <code>loan</code>	<code>CASCADE</code>	An instalment schedule is a weak entity, meaningless without its loan

Q4 (Core)

Anomaly	Definition	In <code>branch_customer_flat</code>
Insert	A fact cannot be recorded without an unrelated fact also existing	The Jaipur branch cannot be registered until it has its first customer, because the table's key is <code>cust_id</code>
Update	One fact stored many times requires many correct updates	Renaming Koramangala changed 1 row of 4; the other three kept their old spellings
Delete	Removing one fact destroys another that was only stored there	Deleting customer 1027's row removed their phone number, city and product list along with the account

Q5

```
BRANCH ||-----o{ EMPLOYEE
```

An employee belongs to exactly one branch, and that branch is mandatory. A branch may have zero, one or many employees. Physically: `employee.branch_id` is `NOT NULL` with a foreign key to `branch`.

```
LOAN ||-----|{ LOAN_PAYMENT
```

A loan payment belongs to exactly one loan, mandatory. A loan has **at least one** payment row and may have many - the `|{` makes the many side mandatory too. Physically: `loan_payment.loan_id` is `NOT NULL`, and the "at least one" rule cannot be expressed as a constraint. It has to be guaranteed by the process that creates a loan, which must create its schedule in the same transaction.

Award a bonus mark for noticing that the mandatory-many side is not enforceable declaratively.

Q6

A foreign key holds **one** value, so it can point at one parent row. In a many-to-many relationship each side needs to reference many rows on the other, and there is no single column that can hold many values. Storing a list in one column is exactly the 1NF violation the `products` column demonstrated.

The junction table's primary key should be the composite of the two foreign keys. That gives uniqueness for free - the same customer cannot be enrolled in the same product twice - and it is the natural key of the relationship. If the relationship itself has a history, such as a customer enrolling, leaving and re-enrolling, the key needs a third column such as `enrolled_on`, or a surrogate key with a partial unique index on the active rows.

Q7 (Core)

Form	Rule	Anomaly removed
1NF	Atomic values, no repeating groups, every row uniquely identifiable	Multiple values cannot be stored, indexed, constrained or counted
2NF	In 1NF, and no non-key column depends on part of a composite key	Parent facts repeated on every child row
3NF	In 2NF, and no non-key column depends on another non-key column	Update anomaly; the same fact stored in many rows diverging

Form	Rule	Anomaly removed
BCNF	Every determinant is a candidate key	Residual redundancy where candidate keys overlap

Q8

A partial dependency means a non-key column depends on **part** of the primary key. A single-column key has no proper parts, so no such dependency can exist. Any 1NF table with a single-column key is automatically in 2NF.

The implication: a schema using surrogate keys throughout skips 2NF concerns entirely, and normalization work reduces to 1NF and 3NF. This is a genuine practical benefit of surrogate keys and is rarely stated. Composite natural keys are where 2NF violations live, and junction tables are the place they most often appear.

Q9 (Core)

Table	Violates	Fix
customer(customer_id, name, phone_1, phone_2, phone_3)	1NF - repeating group	Separate customer_phone(customer_id, phone_type, phone_number)
order_line(order_id, product_id, quantity, product_name, product_price)	2NF - product_name and product_price depend on product_id, part of the composite key	Move them to product, keyed on product_id
employee(employee_id, name, branch_id, branch_name, branch_city)	3NF - branch_name depends on branch_id, a non-key column	Move branch details to branch
account(account_id, customer_id, customer_name, balance)	3NF - customer_name depends on customer_id, a non-key column	Move it to customer

A common wrong answer is calling the second one 3NF. The distinction is that the determinant is part of the primary key, which makes it partial rather than transitive.

Q10 (Core)

It breaks **3NF**, or more precisely it stores a derived value that is not functionally dependent on the key at all - it is computed from another table entirely.

Business reason: a balance enquiry must answer in milliseconds and is called thousands of times a minute, while computing it from scratch reads every transaction the account has ever had. On an account with twenty years of history that is unacceptable.

What must exist alongside it: a **reconciliation control** that periodically compares the stored balance against the computed one and raises an alert on any difference beyond a rounding tolerance. A derived value with no control is a value nobody can trust.

Level 2 - Design and normalize

Q11 (Core)

```
-- branch_code -> branch_ifsc holds: zero branch codes have more than one IFSC
SELECT branch_code, COUNT(DISTINCT branch_ifsc) AS distinct_ifsc
FROM   branch_customer_flat
GROUP BY branch_code
HAVING COUNT(DISTINCT branch_ifsc) > 1;
-- 0 rows

-- branch_code -> branch_name does NOT hold
SELECT branch_code, COUNT(DISTINCT branch_name) AS distinct_names
FROM   branch_customer_flat
GROUP BY branch_code
HAVING COUNT(DISTINCT branch_name) > 1;
```

branch_code	distinct_names
BR03	4

What the second result means. The dependency `branch_code -> branch_name` is supposed to hold - a branch code has exactly one name. The data violates it, which means the data is **corrupt**, not that the dependency is absent.

This is the important distinction. A functional dependency is a statement about the business, and testing it against data can only ever disprove it or fail to disprove it. Here the test surfaced four spellings of one name, which is a data quality defect caused by the design permitting it. **The design allowed the corruption; the query found it.**

Q12

```
SELECT branch_code,
       COUNT(*)           AS customer_rows,
       COUNT(*) - 1       AS redundant_copies,
       SUM(LENGTH(branch_name)) AS bytes_stored,
       MAX(LENGTH(branch_name)) AS bytes_if_stored_once
FROM   branch_customer_flat
GROUP BY branch_code
ORDER BY branch_code;
```

branch_code	customer_rows	redundant_copies
BR01	4	3
BR02	4	3
BR03	4	3
BR04	4	3
BR05	4	3
BR06	4	3
BR07	3	2
BR08	3	2

Bank-wide:

```
SELECT SUM(LENGTH(branch_name)) AS bytes_now FROM branch_customer_flat;           -- 382
SELECT SUM(LENGTH(branch_name)) AS bytes_normalized
FROM   (SELECT DISTINCT branch_code, branch_name FROM branch_customer_flat) d;    -- 134
```

382 bytes against 134, on thirty rows.

Do not let the class conclude that this is about storage. 248 bytes is nothing, and on 30 million customers it would still only be a few hundred megabytes, which is cheap. The cost that matters is the four spellings: **the redundancy is what made the corruption possible.** Storage is the visible symptom; correctness is the actual problem.

Q13

```
SELECT COUNT(*) AS rows, COUNT(DISTINCT cust_phone) AS distinct_phones
FROM   branch_customer_flat;
```

```
rows | distinct_phones
-----+-----
30  |              30
```

Thirty distinct values across thirty rows, so `cust_phone` is unique and is a candidate key **in this data**.

Why it is still not the primary key: a phone number is not stable - customers change numbers, and when they do every child row referencing the customer would have to change with it.

Also creditable: phone numbers get reassigned by the telecom operator to a different person, and a customer may share a landline with a spouse who is also a customer, so uniqueness is an accident of this dataset rather than a business rule.

Q14 (Core)

```
CREATE TABLE ex4_customer_account AS
SELECT cust_id, acct_no_1 AS acct_no, acct_type_1 AS acct_type, balance_1 AS balance
FROM   branch_customer_flat WHERE acct_no_1 IS NOT NULL
UNION ALL
SELECT cust_id, acct_no_2, acct_type_2, balance_2
FROM   branch_customer_flat WHERE acct_no_2 IS NOT NULL;

CREATE TABLE ex4_customer_product AS
SELECT f.cust_id, TRIM(s.product_code) AS product_code
FROM   branch_customer_flat f
      CROSS JOIN LATERAL UNNEST(STRING_TO_ARRAY(f.products, ',')) AS s(product_code)
WHERE  f.products <> 'NONE';

SELECT (SELECT COUNT(*) FROM ex4_customer_account) AS accounts,
       (SELECT COUNT(*) FROM ex4_customer_product) AS product_holdings;
```

```
accounts | product_holdings
-----+-----
36  |              58
```

36 accounts, matching the real `account` table: 30 from the first slot and 6 from the second. 59 product holdings: 27 cards, 20 fixed deposits and 12 loans.

Marking: full marks require `UNION ALL` not `UNION` - `UNION` would silently merge two accounts that happened to share every value.

Q15 (Core)

```
-- Branch identity and the columns that are consistent
CREATE TABLE ex4_branch AS
SELECT DISTINCT branch_code, branch_ifsc, branch_city
FROM   branch_customer_flat;
-- 8 rows

-- The name needs a decision, because BR03 has four
ALTER TABLE ex4_branch ADD COLUMN branch_name VARCHAR(60);

UPDATE ex4_branch b
SET    branch_name = (SELECT f.branch_name
                      FROM   branch_customer_flat f
                      WHERE  f.branch_code = b.branch_code
                      GROUP  BY f.branch_name
                      ORDER  BY COUNT(*) DESC, f.branch_name
                      LIMIT  1);

ALTER TABLE ex4_branch ALTER COLUMN branch_name SET NOT NULL;
ALTER TABLE ex4_branch ADD CONSTRAINT pk_ex4_branch PRIMARY KEY (branch_code);

CREATE TABLE ex4_customer AS
SELECT cust_id, cust_name, cust_phone, cust_city, branch_code
FROM   branch_customer_flat;

ALTER TABLE ex4_customer ADD CONSTRAINT pk_ex4_customer PRIMARY KEY (cust_id);
ALTER TABLE ex4_customer ADD CONSTRAINT fk_ex4_customer_branch
    FOREIGN KEY (branch_code) REFERENCES ex4_branch(branch_code);
```

The rule applied: most frequent spelling, ties broken alphabetically. For BR03 all four spellings occur exactly once, so the tie-break decides and `Koramangala` survives.

State the limitation. "Most frequent" is a heuristic with no authority behind it. Had the corrupt spelling been the common one, this rule would have enshrined it. In a real cleanup the authoritative name comes from the branch licence register or the RBI IFSC directory, and the query is used only to **find** the discrepancies, never to resolve them.

Marking: deduct if the learner does not acknowledge this. The technical answer is easy; the governance judgement is the point.

Q16 (Core)

```
-- INSERT anomaly gone: a branch can exist with no customers
BEGIN;
INSERT INTO ex4_branch (branch_code, branch_ifsc, branch_city, branch_name)
VALUES ('BR09', 'ICIC0000709', 'Jaipur', 'Malviya Nagar');
SELECT COUNT(*) FROM ex4_branch;           -- 9
ROLLBACK;
```

```
-- UPDATE anomaly gone: one fact, one row, one update
BEGIN;
UPDATE ex4_branch SET branch_name = 'Koramangala Main' WHERE branch_code = 'BR03';
-- UPDATE 1
SELECT branch_code, branch_name FROM ex4_branch WHERE branch_code = 'BR03';
ROLLBACK;

-- DELETE anomaly gone: removing an account leaves the customer intact
BEGIN;
DELETE FROM ex4_customer_account WHERE acct_no = 'ICIC0000009032';
SELECT cust_id, cust_name, cust_phone FROM ex4_customer WHERE cust_id = 1027;
ROLLBACK;
```

The third query still returns Gaurav Patel with his phone number. In the flat table the same operation erased him.

Also accept a fourth demonstration: attempting to insert a customer at a non-existent branch now fails on `fk_ex4_customer_branch`, which is a guarantee the flat table could not offer at all.

Q17 (Core)

```
CREATE TABLE ex4_locker (
    locker_id    INTEGER    NOT NULL,
    branch_id    INTEGER    NOT NULL,
    locker_number VARCHAR(10) NOT NULL,
    locker_size   VARCHAR(10) NOT NULL,
    annual_rent   NUMERIC(10,2) NOT NULL,
    status        VARCHAR(12) DEFAULT 'AVAILABLE' NOT NULL,
    CONSTRAINT pk_ex4_locker PRIMARY KEY (locker_id),
    CONSTRAINT uq_ex4_locker UNIQUE (branch_id, locker_number),
    CONSTRAINT fk_ex4_locker_branch FOREIGN KEY (branch_id)
        REFERENCES branch(branch_id) ON DELETE RESTRICT,
    CONSTRAINT ck_ex4_locker_size CHECK (locker_size IN ('SMALL','MEDIUM','LARGE')),
    CONSTRAINT ck_ex4_locker_rent CHECK (annual_rent > 0),
    CONSTRAINT ck_ex4_locker_status CHECK (status IN ('AVAILABLE','ALLOTTED','SURRENDERED'))
);

CREATE TABLE ex4_locker_agreement (
    agreement_id INTEGER    NOT NULL,
    locker_id     INTEGER    NOT NULL,
    customer_id   INTEGER    NOT NULL,
    start_date    DATE       NOT NULL,
    end_date      DATE       NULL,
    status        VARCHAR(12) DEFAULT 'ACTIVE' NOT NULL,
    CONSTRAINT pk_ex4_locker_agreement PRIMARY KEY (agreement_id),
    CONSTRAINT fk_ex4_la_locker FOREIGN KEY (locker_id)
        REFERENCES ex4_locker(locker_id) ON DELETE RESTRICT,
    CONSTRAINT fk_ex4_la_customer FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id) ON DELETE RESTRICT,
    CONSTRAINT ck_ex4_la_dates CHECK (end_date IS NULL OR end_date > start_date),
    CONSTRAINT ck_ex4_la_status CHECK (status IN ('ACTIVE','CLOSED','SURRENDERED'))
);

CREATE TABLE ex4_locker_visit (
    visit_id      BIGINT    NOT NULL,
    agreement_id  INTEGER    NOT NULL,
    visited_at    TIMESTAMP NOT NULL,
```

```

supervised_by INTEGER NOT NULL,
CONSTRAINT pk_ex4_locker_visit PRIMARY KEY (visit_id),
CONSTRAINT fk_ex4_lv_agreement FOREIGN KEY (agreement_id)
    REFERENCES ex4_locker_agreement(agreement_id) ON DELETE RESTRICT,
CONSTRAINT fk_ex4_lv_employee FOREIGN KEY (supervised_by)
    REFERENCES employee(employee_id) ON DELETE RESTRICT
);

```

The design decisions worth marking.

`ex4_locker` exists independently of any allotment, which satisfies the requirement that an unallotted locker is still recorded. A design that put `customer_id` directly on the locker would fail that requirement and would also lose all allotment history.

The visit log references the **agreement**, not the locker. That is deliberate: it records who was entitled to open it at the time, which is what an investigation needs. Referencing the locker alone would leave you unable to say which customer's visit it was after the locker is re-allotted.

`ON DELETE RESTRICT` on `supervised_by` means an employee who has supervised a visit cannot be deleted from the system. That is correct - the audit trail must survive the employee leaving. The alternative is to soft-close employees rather than delete them, which is what a bank actually does.

Q18

Question	Answer
Which entity is weak	<code>ex4_locker_visit</code> . A visit has no meaning without the agreement it occurred under, and would be identified as "visit n of agreement x"
Which relationship is many-to-many, and how resolved	Customer to locker, over time. Resolved by <code>ex4_locker_agreement</code> , which carries the relationship's own attributes: start date, end date and status
Which foreign key is nullable and why	None of the foreign keys. The nullable column is <code>end_date</code> , because an active agreement genuinely has no end date yet, and NULL is the correct representation of a fact that does not exist
Two business rules as <code>CHECK</code> constraints	<code>end_date IS NULL OR end_date > start_date</code> , and <code>annual_rent > 0</code> . Also acceptable: the locker size and status domains

The rule that cannot be a `CHECK`: "a locker may be allotted to only one customer at a time". That is a constraint across **several rows** of `ex4_locker_agreement`, and `CHECK` sees only one row. It needs either a unique index on the active rows, or a trigger.

```

-- PostgreSQL: a partial unique index enforces one active agreement per locker
CREATE UNIQUE INDEX uq_ex4_la_active
    ON ex4_locker_agreement (locker_id) WHERE status = 'ACTIVE';

```

Award a bonus mark for spotting this. It is the most common gap in a first schema design: rules that span rows cannot be expressed as `CHECK` constraints.

Q19 (Core)

The constraint is already declared as `ck_ex4_la_dates`. Prove it:

```
INSERT INTO ex4_locker_agreement
  (agreement_id, locker_id, customer_id, start_date, end_date, status)
VALUES (9001, 1, 1001, DATE '2026-04-01', DATE '2026-03-01', 'ACTIVE');
```

```
ERROR: new row for relation "ex4_locker_agreement" violates check constraint
"ck_ex4_la_dates"
```

Then show the NULL case still works, because an active agreement has no end date:

```
INSERT INTO ex4_locker_agreement
  (agreement_id, locker_id, customer_id, start_date, end_date, status)
VALUES (9002, 1, 1001, DATE '2026-04-01', NULL, 'ACTIVE');
-- INSERT 0 1
```

The `end_date IS NULL OR ...` clause is essential. Written as `end_date > start_date` alone, the constraint would evaluate to NULL for an open agreement - and a **CHECK** that evaluates to NULL **passes**, which is a subtlety worth stating explicitly. **CHECK** rejects only on FALSE.

Q20

```
CREATE TABLE ex4_branch_daily_summary (
  branch_id      INTEGER      NOT NULL,
  summary_date   DATE         NOT NULL,
  txn_count      INTEGER      NOT NULL,
  total_credits  NUMERIC(15,2) NOT NULL,
  total_debits   NUMERIC(15,2) NOT NULL,
  CONSTRAINT pk_ex4_bds PRIMARY KEY (branch_id, summary_date),
  CONSTRAINT fk_ex4_bds_branch FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);

INSERT INTO ex4_branch_daily_summary
  (branch_id, summary_date, txn_count, total_credits, total_debits)
SELECT a.branch_id,
  CAST(t.txn_date AS DATE),
  COUNT(*),
  SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END),
  SUM(CASE WHEN t.txn_type = 'DEBIT' THEN t.amount ELSE 0 END)
FROM   account_txn t
JOIN   account a ON a.account_id = t.account_id
WHERE  t.status = 'SUCCESS'
  AND  t.txn_date >= DATE '2026-01-01'
  AND  t.txn_date < DATE '2026-02-01'
GROUP BY a.branch_id, CAST(t.txn_date AS DATE);
```

The primary key is `(branch_id, summary_date)` - a composite natural key, which is correct here because the grain of the table is branch plus date, and a surrogate would add nothing. It also prevents the summary job from double-loading a day, which is the failure mode these tables actually suffer from.

The closing deposit total was in the requirement but is deliberately omitted: `account.balance` is a current value with no history, so it cannot be reconstructed for a past date from the tables available. **Say so rather than inventing it.** Capturing it would require the summary job to snapshot balances each night going forward.

Award full marks for either implementing it as a nightly snapshot or explicitly identifying that it cannot be backfilled. Deduct for silently computing something that looks like a historical balance but is not.

Q21 (Core)

```
SELECT s.branch_id, s.summary_date,
       s.txn_count      AS summary_count,
       t.txn_count      AS live_count,
       s.total_credits  AS summary_credits,
       t.total_credits  AS live_credits
FROM   ex4_branch_daily_summary s
FULL OUTER JOIN (
    SELECT a.branch_id,
           CAST(t.txn_date AS DATE) AS summary_date,
           COUNT(*) AS txn_count,
           SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END) AS total_credits
    FROM   account_txn t JOIN account a ON a.account_id = t.account_id
    WHERE  t.status = 'SUCCESS'
           AND t.txn_date >= DATE '2026-01-01' AND t.txn_date < DATE '2026-02-01'
    GROUP BY a.branch_id, CAST(t.txn_date AS DATE)
) t ON t.branch_id = s.branch_id AND t.summary_date = s.summary_date
WHERE  s.txn_count IS DISTINCT FROM t.txn_count
OR     ABS(COALESCE(s.total_credits, 0) - COALESCE(t.total_credits, 0)) > 0.01;
```

Zero rows: the summary agrees with the source.

A non-empty result means one of four things, and the shape of the row tells you which: a day present in the summary but not in the source, meaning the summary has rows it should not; a day in the source but not the summary, meaning the load missed it; matching days with different counts, meaning transactions arrived after the summary ran; or matching counts with different totals, meaning a rounding or type difference in the load.

Two details worth teaching. `FULL OUTER JOIN` rather than an inner join, because an inner join can only find rows that exist on both sides and would silently miss an entirely absent day - the most likely failure. And `IS DISTINCT FROM` rather than `<>`, because `<>` returns NULL when one side is NULL, so a missing day would fail the comparison and be dropped from the exception report.

Level 3 - Judgement

Q22

Model answer.

To the vendor:

We cannot accept `ON DELETE CASCADE` on `account_txn` referencing `account`, or on `account` referencing `customer`. With both in place, a single `DELETE` of one customer row removes that customer's accounts and every transaction those accounts have ever carried. The statement succeeds, returns no error, and leaves no record of what was removed. There is no recovery path short of a point-in-time restore, and no way to discover the loss until a reconciliation or a regulator asks for the history.

The scenario we are protecting against is not malice. It is a support engineer running a customer cleanup script at 2 a.m. during an incident, with a `WHERE` clause that is slightly wider than intended. `RESTRICT` turns that into an error message. `CASCADE` turns it into an outage nobody notices for a week.

We require `ON DELETE RESTRICT` or `NO ACTION` on every foreign key referencing a financial record: accounts, transactions, loans, payments, deposits and statements. `CASCADE` is acceptable on tables that are genuinely owned by their parent and carry no financial or audit significance - preferences, notification settings and reward enrolments. We are happy to review the list table by table.

This is not negotiable for us because our transaction retention obligation is seven years, and "the database was kept clean" is not a defence we can offer a regulator.

Marking: 5 marks. Look for the specific scenario, the distinction between financial and preference data, a concrete requirement rather than a general objection, and a tone that leaves the vendor relationship intact.

Q23 (Core)

Two arguments against.

1. **The limit is arbitrary and will be wrong.** Three columns is a guess encoded in the schema. When a customer needs a fourth, the fix is an `ALTER TABLE` on a production table plus a change to every query, screen and report that reads phone numbers. A separate table absorbs the change with no schema modification at all.
2. **You cannot query it properly.** "Which customers have a verified mobile number?" requires checking three columns with three `OR` branches, and there is nowhere to record which of the three is the mobile, which is verified, or which is preferred. Those attributes belong to the phone number, and there is no room for them in a bare column.

Where their position is reasonable. If the bank's business rule genuinely is "exactly one registered mobile and optionally one landline", with fixed, distinct meanings for each and no attributes beyond the number itself, then `mobile_number` and `landline_number` are two different attributes of the customer, not a repeating group. That is not a 1NF violation - it is two separate facts that happen to look similar. The test is whether the columns are **interchangeable**: `phone_1`, `phone_2`, `phone_3` are, and that is what makes them a repeating group.

Award full marks for the interchangeability test. It is the distinction most learners miss.

Q24

Why it is acceptable in the warehouse. The fact table is loaded once, by a controlled ETL job, and is never updated afterwards. Redundancy only causes an update anomaly when the redundant value can be **updated inconsistently**, and here nothing updates it. The table is also read by aggregate queries that would otherwise join 400 million rows to a dimension on every run, so removing the join is a large and measurable win.

Why it is not acceptable in customer. That table is updated continuously, by many applications and by branch staff. A branch name stored there can be changed in one row and not another, which is precisely what produced four spellings of Koramangala.

The control that makes it safe: the ETL job is the single writer, and it derives `branch_name` from the branch dimension at load time. Nothing else can write the column, and a rename in the dimension is applied by re-running the load or by a controlled bulk update. Additionally, the warehouse row is a **point-in-time record**: if the branch is renamed next year, the fact row arguably should keep the name the branch had when the transaction happened, which is a slowly changing dimension, covered in Module 10.

Q25 (Core)

Model answer.

I would not accept it as a blanket rule, but I would accept a version of it.

The problem with automatic most-frequent selection is that it has no authority behind it. It happens to give the right answer for BR03 only because the correct spelling was one of the four; had three branches been recorded consistently wrong and once right, the rule would enshrine the error and we would have no record that a decision was ever made.

What I would propose instead, and it costs less than a day: run the discrepancy query across all 8 branches to produce the exception list. Where the variants differ only by whitespace or case, resolve them automatically - that is a normalization of formatting, not a choice between competing facts, and it is safe. Where they differ in substance, match against the RBI IFSC directory, which is authoritative, freely available, and keyed on the IFSC code we already have and which the query confirmed is consistent. Only the residue goes to a human.

Whatever we do, the cleanup must write a mapping table recording the original value, the chosen value and the reason. That is what lets someone answer "why does this branch have this name" in two years, and it is what turns a bulk update into something an auditor can accept.

If the deadline genuinely does not allow even that, I would rather ship with the branch names as they are and the foreign key in place, and clean the names in a second release, than silently pick values and lose the evidence.

Q26

Model answer.

The argument is half right, and taking it seriously is the correct instinct. Joins on indexed keys **are** cheap, storage **is** cheap, and there are real systems where denormalization is the right answer - this bank runs one, in the reporting warehouse.

But the premise misidentifies what normalization is for. **It is not a storage optimisation.** The 248 bytes saved by extracting the branch name in this dataset is irrelevant, and would still be irrelevant at 30 million customers.

The cost normalization prevents is **incorrect data**. When one fact is stored in four places, keeping the four in agreement becomes the responsibility of every piece of code that writes any of them, forever, including code that has not been written yet and code written by people who do not know the other three copies exist. Sooner or later one write misses one copy, and from that moment the database contains two contradictory answers to the same question with nothing to indicate which is right.

That is what happened here. Four spellings of Koramangala meant every report grouped by branch name showed the branch four times with a quarter of its real figures, and nobody noticed until an audit. No amount of hardware fixes it, because it is not a performance problem. Storing the fact once makes the inconsistency **impossible** rather than merely unlikely, and that is a guarantee, not an optimisation.

The honest version of the colleague's argument is: where a table is written by exactly one controlled process and never updated afterwards, redundancy is safe and can be worth it. That is true, it describes the warehouse, and it is why denormalization is a legitimate technique with a specific precondition rather than a general licence.

Marking: 5 marks. Full marks require conceding the valid part, identifying correctness rather than storage as the real cost, and naming the precondition under which the colleague is right.

Q27

Model answer.

Dear Mr Iyer,

Ticket ICICI-4577 is closed. Here is what was wrong and what has replaced it.

The customer master held everything about a customer on a single wide row: their personal details, their branch's name and address, up to two of their accounts, and a text list of the products they hold. Because each customer's row carried its own copy of the branch address, the branch address was stored once per customer rather than once per branch. Nothing made those copies agree with each other, and over time they stopped agreeing - Koramangala was recorded four different ways by four different people.

Both audit findings come from that single design decision. The spellings diverged because the same fact was typed in many places. Closing an account deleted the customer's phone number because the account and the customer were the same row, so there was nowhere else the phone number was kept.

The replacement stores each fact exactly once, in a table about the thing it describes. A branch address lives in the branch record. A customer's phone number lives in the customer record. An account lives in its own record and points at its owner. Because the branch address now exists in only one place, it is not merely unlikely that two spellings will appear - it is not possible. And because closing an account removes only the account record, the customer's details are untouched by definition.

Two things change for the branches. Branch details are maintained centrally and can no longer be edited from a customer screen, which is deliberate. And a customer may now hold any number of accounts and products rather than two and a text list, so the account screens will show a list rather than fixed fields.

One thing I want to flag. Fixing the design stops new divergence; it does not decide which of the four Koramangala spellings was correct. We have used the branch licence register as the authority and recorded the mapping, so any question about a historical report can be answered.

Marking: 5 marks. Look for a plain-English explanation of "stored once", both findings traced to one root cause, the shift from unlikely to impossible stated explicitly, a genuine operational consequence for the branches, and the honest note that the design fix does not clean historical data. Deduct for any use of the forbidden vocabulary.